Manual: como rodar e apresentar o Conecta Vida no Docker Swarm

Projeto **Conecta Vida**, IFSP Bragança Paulista. Repositório: https://github.com/Conecta-Vida/Docker-Teste

Parte A: o que o professor pediu e onde está

Requisito	Como atendemos	Onde mostrar
1. Dockerfiles (imagens) dos serviços	4 imagens multi-stage em Alpine Linux: `admin-api`, `mobile-api` (Java 21), `admin-web` (React + Nginx) e `mobile-web` (Flutter Web + Nginx)	Arquivos `admin-api/Dockerfile`, `mobile-api/Dockerfile`, `admin-web/Dockerfile` e `mobile-app/Dockerfile`
2. Docker Swarm com docker-in-docker	Cluster de **3 nós**: o PC é o manager e `worker-1` e `worker-2` são containers `docker:dind`	Visualizer e `docker node ls`
3. Docker Compose (infraestrutura como código), variação de réplicas e Visualizer	`docker-compose.yml` descreve os 6 serviços, a rede, o volume e as réplicas. As réplicas mudam pelo arquivo (`replicas`) e pelo terminal (`docker service scale`)	`docker-compose.yml` e o Visualizer em http://localhost:8085

Os 6 serviços da stack:

Serviço	O que é	Porta	Réplicas	Onde roda
`postgres`	Banco de dados	5432	1	Manager (fixo)
`admin-api`	API do painel do gestor	8080	2	Qualquer nó
`mobile-api`	API do app do cidadão	8081	3	Qualquer nó
`visualizer`	Painel gráfico do cluster	8085	1	Manager (fixo)
`admin-web`	Site do painel admin	8088	2	Qualquer nó
`mobile-web`	App do cidadão no navegador	8089	2	Qualquer nó

Parte B: preparar o computador (uma vez só)

B1. Instale o **Docker Desktop**: https://www.docker.com/products/docker-desktop/

B2. Instale o **Git for Windows**, que traz o **Git Bash**: https://git-scm.com/download/win

B3. Abra o Docker Desktop e espere o ícone da baleia ficar parado, sem a mensagem "starting".

B4. Abra o **Git Bash** e confira se o Docker responde:

```
docker --version
```

```
docker compose version
```

B5. Baixe o projeto:

```
cd ~
```

```
git clone https://github.com/Conecta-Vida/Docker-Teste.git
```

```
cd Docker-Teste
```

> Todos os comandos deste manual são executados **no Git Bash**, dentro da pasta `~/Docker-Teste`. No PowerShell os scripts `.sh` não funcionam.

Parte C: subir o projeto

C1. Atualize o projeto. Faça isso sempre antes de apresentar:

```
cd ~/Docker-Teste && git pull
```

C2. Se já rodou uma versão antiga antes, limpe tudo:

```
./infra/swarm/seminario.sh derrubar --apagar-dados
```

C3. Suba o cluster:

```
./infra/swarm/seminario.sh subir
```

O script faz tudo sozinho:

1. Ativa o Docker Swarm.
2. Constrói as 4 imagens. **Na primeira vez leva de 10 a 15 minutos**, porque baixa o Java, o Node e o Flutter. Depois fica rápido.
3. Cria o registry local e envia as imagens para ele.
4. Cria os workers `worker-1` e `worker-2` (Docker-in-Docker).
5. Conecta os workers ao manager e etiqueta os nós.
6. Faz o deploy: `docker stack deploy -c docker-compose.yml conectavida`.

C4. Espere cerca de **1 minuto** e confira:

```
./infra/swarm/seminario.sh status
```

Está pronto quando:

- os 3 nós aparecem como `Ready`;

- todos os serviços mostram réplicas completas (`1/1`, `2/2`, `3/3`).

C5. Abra no navegador:

O quê	Endereço	Login
Visualizer	http://localhost:8085	(sem login)
Painel admin	http://localhost:8088	`admin@conectavida.com` / `admin123`
App do cidadão	http://localhost:8089	`cidadao@conectavida.com` / `cidadao123`
Documentação das APIs	http://localhost:8080/swagger-ui.html	(sem login)

> Para o app parecer de celular: aperte **F12** no Chrome e clique no ícone de celular (Ctrl+Shift+M).

Parte D (opcional): usar os dados reais do Supabase

O banco começa com dados de demonstração. Para trazer os dados que estavam no Supabase:

D1. Pegue a senha atual do banco em **Supabase → Project Settings → Database**.

D2. Com a stack rodando, no Git Bash:

```
export SUPABASE_DB_URL='postgresql://postgres.pborxjuuexiqollesfxw@aws-1-sa-east-1.pooler.supabase.com:5432/postgres'
```

```
export SUPABASE_DB_PASSWORD='cole-a-senha-aqui'
```

```
./infra/postgres/importar-supabase.sh
```

D3. Para os **logins antigos** funcionarem, o salt precisa ser o mesmo de antes. Ele está no `app.security.salt` dos `application.properties` antigos. Antes do `subir`, rode:

```
export CONECTA_API_SALT='valor-do-salt-antigo'
```

> O script só **lê** do Supabase e não altera nada lá. Se apagar os dados com `derrubar --apagar-dados`, rode a importação de novo.

Parte E: preparar a apresentação (5 minutos antes)

E1. Confirme que tudo está no ar (Parte C4).

E2. Organize a tela:

- **Navegador** com 3 abas: Visualizer (8085), painel (8088) e app (8089).
- **VS Code** com estes arquivos abertos: `admin-api/Dockerfile`, `admin-web/Dockerfile` e `docker-compose.yml`.
- **Git Bash** com a fonte grande (Ctrl + roda do mouse).

E3. Faça login no painel e no app antes, para não perder tempo.

Parte F: roteiro da apresentação (cerca de 12 minutos)

F1. Abertura (1 min)

> "O Conecta Vida centraliza informações de saúde pública: o gestor publica campanhas e informativos pelo painel web e o cidadão recebe pelo aplicativo. Hoje vamos mostrar como colocamos o sistema inteiro em containers e orquestramos com o Docker Swarm."

F2. Requisito 1: Dockerfiles (3 min, no VS Code)

Abra o `admin-api/Dockerfile` e explique:

> "Este Dockerfile é **multi-stage**. O primeiro estágio, `FROM eclipse-temurin:21-jdk-alpine AS build`, tem o JDK completo e compila o projeto com o Gradle. O segundo, `FROM eclipse-temurin:21-jre-alpine`, tem só o Java de execução e copia **apenas o `app.jar`**. A imagem final fica sem compilador nem código-fonte, roda em **Alpine Linux**, com um **usuário sem root** e um **healthcheck** que o Swarm usa para saber se a réplica está saudável."

Abra o `admin-web/Dockerfile`:

> "O front segue a mesma ideia: o Node compila o React e o Nginx entrega o site. A imagem final tem cerca de 74 MB. O Nginx também repassa as chamadas `/api` para a API dentro do cluster."
>
> "No total são 4 imagens nossas: as duas APIs, o painel e o app em versão web."

F3. Iniciar o roteiro guiado

No Git Bash:

```
./infra/swarm/seminario.sh apresentar
```

O script mostra **em amarelo o que falar**, executa cada demonstração e espera você apertar **Enter**.

F4. Requisito 2: Swarm (passos 1 e 2 do script, 2 min)

- **Passo 1, os nós:**

> "Temos 3 nós. O **manager** coordena o cluster: decide onde cada container roda e recria o que falhar. Os **workers** executam os containers. Usamos **Docker-in-Docker**: cada worker é um container que roda um Docker dentro dele, simulando outra máquina."

- **Passo 2, o Visualizer:**

  > "O banco fica **fixo no manager** (`constraint: node.role == manager`), porque guarda dados. As APIs e os sites não guardam estado, então o Swarm **espalha as cópias** entre os nós. Caixas da mesma cor são réplicas do mesmo serviço."

F5. O sistema funcionando (passo 3, 2 min)

- No **painel**, mostre o dashboard e, em **Campanhas**, crie uma campanha nova (ex.: "Vacinação contra a gripe").
- No **app**, atualize a página e mostre a campanha aparecendo para o cidadão, que pode se inscrever.

  > "As duas telas conversam com APIs que estão rodando em vários nós do cluster, ao mesmo tempo."

F6. Requisito 3: Compose e variação de réplicas (passos 4 e 5, 3 min)

Antes do passo 4, mostre no VS Code o `docker-compose.yml`:

> "Isto é **infraestrutura como código**: os 6 serviços, a rede overlay, o volume do banco, os limites de memória e o número de réplicas estão descritos neste arquivo. Vejam a linha `replicas: ${MOBILE_API_REPLICAS:-3}`: o padrão são 3 cópias."

- **Passo 4:** o script reaplica o arquivo com 6 réplicas.

  > "Olhem o Visualizer: as caixas da mobile-api passaram de 3 para 6, distribuídas entre os nós."

- **Ainda no passo 4:** no próximo Enter, o script volta para 2 réplicas e as caixas somem.
- **Passo 5:** o script usa `docker service scale`.

  > "Também dá para escalar direto pelo terminal. O arquivo continua sendo a referência: no próximo deploy, vale o que está nele."

F7. Extra: alta disponibilidade (passo 6, 1 min)

> "E se uma máquina cair?"

O script desliga o `worker-2`. Em cerca de 30 segundos ele fica vermelho no Visualizer e as réplicas dele reaparecem nos outros nós. Atualize o painel para mostrar que o sistema continua funcionando. No próximo Enter o worker é religado.

F8. Fechamento (passo 7, 30 s)

> "Atendemos os três requisitos: **Dockerfiles multi-stage** para 4 serviços, **cluster Swarm com 3 nós em Docker-in-Docker**, e **Docker Compose como infraestrutura como código**, variando réplicas e acompanhando tudo pelo **Docker Visualizer**. Obrigado!"

Parte G: divisão para 3 apresentadores

Quem	Partes	Tempo	Fica no controle de
Pessoa 1	Abertura e **Requisito 1: Dockerfiles** (F1 e F2)	~4 min	VS Code (mostra `admin-api/Dockerfile` e `admin-web/Dockerfile`)
Pessoa 2	**Requisito 2: Swarm** e sistema funcionando (F3, F4 e F5)	~4 min	Git Bash (`apresentar`, passos 1 a 3) e navegador (painel e app)
Pessoa 3	**Requisito 3: Compose e réplicas**, queda de máquina e fechamento (F6, F7 e F8)	~4 min	VS Code (`docker-compose.yml`) e Git Bash (passos 4 a 7)

Dicas:

- Um só computador e um só terminal: quem está falando pede para o colega apertar Enter, ou assume o teclado na sua vez.
- A Pessoa 2 já deixa o painel e o app logados antes de começar.
- Se perguntarem sobre o registry: "É um depósito de imagens dentro do cluster. Construímos as imagens no manager e os workers baixam de lá." Para mostrar: `curl http://localhost:5000/v2/_catalog`.

Parte H: comandos avulsos (se quiser fazer na mão)

Para	Comando
Ver nós, serviços e réplicas	`./infra/swarm/seminario.sh status`
Mudar réplicas pelo compose	`./infra/swarm/seminario.sh replicas mobile-api 6`
Escalar pelo terminal (6/4/2)	`./infra/swarm/seminario.sh escalar`
Escalar um serviço específico	`docker service scale conectavida_mobile-api=6`
Desligar o worker-2	`./infra/swarm/seminario.sh falha`
Religar o worker-2	`./infra/swarm/seminario.sh recuperar`
Ver os nós	`docker node ls`
Ver onde cada réplica está	`docker stack ps conectavida`
Ver o log de uma API	`docker service logs conectavida_admin-api`

Parte I: se algo der errado

Problema	Solução
O Windows pergunta "com qual aplicativo abrir o .sh"	Você está no PowerShell. Use o **Git Bash**.
`docker: command not found` ou erro de conexão	Abra o Docker Desktop e espere ficar pronto.
Alguma caixa vermelha logo depois do `subir`	Normal: as APIs reiniciam até o banco ficar pronto. Espere 1 minuto.
Painel diz "Não foi possível conectar à API"	Espere 1 minuto e confira com `./infra/swarm/seminario.sh status`.
Os workers não entram no cluster	Rode `docker swarm leave --force` e depois `./infra/swarm/seminario.sh subir`.
Depois do `falha`, o worker não volta	`./infra/swarm/seminario.sh recuperar`
Quer recomeçar do zero	`./infra/swarm/seminario.sh derrubar --apagar-dados` e depois `./infra/swarm/seminario.sh subir`
Importação do Supabase: "password authentication failed"	Usuário `postgres.pborxjuuexiqollesfxw`, porta 5432 e senha atual em `SUPABASE_DB_PASSWORD`.
Nada funciona na hora da apresentação	Mostre o vídeo em `documentacao/midia/` e as capturas de tela.

Parte J: depois da apresentação

Para desligar tudo e liberar memória:

```
./infra/swarm/seminario.sh derrubar
```

Para sair do modo Swarm também:

```
docker swarm leave --force
```

Mais detalhes técnicos (cada Dockerfile e o compose explicados bloco a bloco): `documentacao/DOCKER_SWARM.md`.